

HEXA — A-Z BUILD DOCUMENT FOR CLAUDE CODE

Last updated: May 25, 2026 **Target platforms:** iOS 16+ and Android 9+ (API 28+) **Stack:** Expo SDK 51+ (React Native 0.74+), Supabase, MapLibre GL, TypeScript

For Claude Code: This document is the complete product, technical, and execution spec for the Hexa app. Treat each PHASE as a discrete milestone. Do not skip ahead. Do not add features that are not in the current phase. At the end of each phase, run the acceptance checklist before proceeding. Always ask for clarification if a requirement is ambiguous rather than guessing.

TABLE OF CONTENTS

1. Product Overview & Game Design
 2. Core Game Mechanics — The Logic Bible
 3. Technical Architecture
 4. Database Schema (Complete)
 5. Tech Stack & Dependencies
 6. Project Structure
 7. PHASE 0 — Pre-Build Setup (Day 0-3)
 8. PHASE 1 — Foundation & Auth (Week 1)
 9. PHASE 2 — Map & Hex Display (Week 2)
 10. PHASE 3 — GPS, Location & Hex Generation (Week 3)
 11. PHASE 4 — Capture Mechanics (Week 4-5)
 12. PHASE 5 — Points, Levels & Safe Points (Week 6)
 13. PHASE 6 — Medals, Streaks & Gamification (Week 7)
 14. PHASE 7 — Social: Friends, Leaderboards, Sharing (Week 8)
 15. PHASE 8 — Anti-Cheat Layer (Week 9)
 16. PHASE 9 — Polish, Onboarding & Beta (Week 10)
 17. PHASE 10 — Paths System (Month 3)
 18. PHASE 11 — Clans (Month 4)
 19. PHASE 12 — Brand Bazaar & Monetization (Month 4-5)
 20. Appendix A — Capture Flow Pseudocode
 21. Appendix B — Hex Pre-Generation Script
 22. Appendix C — Battery & Performance Targets
 23. Appendix D — Decisions Still Pending
-

1. PRODUCT OVERVIEW & GAME DESIGN

1.1 The 30-Second Pitch

Hexa turns Bangalore into a board game. Every neighbourhood is split into ~150m hexagons. Walk to a hex, stand for 20 seconds, you own it. Hold hexes to earn passive points. Friends and strangers try to steal them. Climb your neighbourhood leaderboard. Earn medals. Compete in clans (your apartment tower vs the next). Eventually, redeem points for sponsored vouchers from local cafés and brands.

1.2 Target Users

- **Primary (60%):** Gen-Z / millennial Bangaloreans aged 22-35, tech workers, walking culture nascent
- **Secondary (25%):** Run-club crowd aged 25-40, Strava-adjacent
- **Tertiary (15%):** 40+ neighbourhood walkers, retired uncles/aunties

1.3 Core Loop

1. Open app → see your colour spread across hexes around you
2. Walk → capture new hexes (20-sec dwell each)
3. Get push notification when someone steals your hex
4. Walk back → reclaim + steal theirs
5. End of day → share capture card to WhatsApp/IG
6. End of week → check leaderboard rank in your pincode
7. End of month → season resets, fresh competition

1.4 Design Principles

- **Simple by default, deep on opt-in.** Aunty sees a map; gen-Z sees clans + medals + leaderboards
- **The map is the reward.** Make the visual game state itself dopamine-worthy
- **Loss aversion > gain motivation.** Push notifications focus on "you lost X" not "earn X"
- **Hyperlocal first.** First 6 months: Bangalore only. Within Bangalore: HSR, Koramangala, Indiranagar only
- **No crypto. No tokens. No NFTs. No user buy-in.** Brand-funded rewards or no rewards
- **Battery is sacred.** Background GPS is the #1 reason users uninstall — design around it from day 1

1.5 What This App Is NOT

- Not a step counter (StepSetGo owns that)
- Not a running app (Strava/INTVL own that)

- Not a fitness app (Cult/Fitbit own that)
 - Not a crypto/M2E app (STEPN died)
 - Not a global app (focus is hyperlocal)
-

2. CORE GAME MECHANICS — THE LOGIC BIBLE

This section is the source of truth for all game logic. Any conflict between this section and the code defaults to this section.

2.1 Hexes — The Building Blocks

Hex grid system: Uber H3, resolution 10

- Average hex area: ~15,047 m² (roughly a small city block)
- Average edge length: ~66m
- Hex diameter (across): ~130m
- Library: `h3-js` (client) + `h3-pg` (server PostgreSQL extension)

Hex properties (stored at generation):

- `h3_index` (bigint, primary key) — H3 cell ID at res 10
- `centroid_lat`, `centroid_lng` (float8) — for map rendering
- `boundary_geojson` (jsonb) — the hex outline polygon
- `pincode` (text) — Bangalore pincode this hex falls in (560001-560103)
- `neighbourhood` (text) — derived from pincode + reverse geocode (e.g., "HSR Sector 7", "Indiranagar")
- `rarity` (int, 1-5, default 1) — 1=common, 5=ultra-rare, set during seeding
- `ip_value` (int, default 100) — Instant Points awarded on capture, range 50-250
- `pph_value` (int, default 3) — Passive Points per Hour, range 1-9
- `is_water` (bool, default false) — lake/tank hexes get special medal eligibility
- `is_park` (bool, default false) — park hexes get +20% IP
- `poi_type` (text, nullable) — for Power Zones (cafe, gym, temple, college)
- `is_power_zone` (bool, default false) — sponsored 3x PPH zones (post-MVP)
- `power_zone_sponsor` (text, nullable) — brand name (post-MVP)

Rarity-to-points formula (deterministic, applied at seeding):

- Rarity 1 (90% of hexes): IP=100, PPH=3
- Rarity 2 (7%, near minor POIs): IP=130, PPH=4
- Rarity 3 (2%, near major POIs/parks): IP=170, PPH=5
- Rarity 4 (0.9%, landmarks like Vidhana Soudha, MG Road, Cubbon Park): IP=210, PPH=7

- Rarity 5 (0.1%, "Crown Hexes" — 1 per pincode, e.g., the geometric center): IP=250, PPH=9

2.2 Capture Mechanic (THE CORE)

To capture an unowned hex:

1. User must be physically inside the hex (point-in-polygon check against H3 boundary)
2. GPS accuracy must be ≤ 25 meters (reject if worse, show "improve GPS signal")
3. User must remain inside for **20 continuous seconds** (the "dwell timer")
4. During those 20 seconds:
 - GPS speed must stay ≤ 10 km/h (if user exceeds, dwell timer pauses)
 - Accelerometer must register ≥ 2 step events (proves device is moving with a human, not stationary spoofed)
 - User must not leave the hex boundary (if they do, timer resets to 0)
5. On successful capture:
 - Award IP value to user
 - Write capture record to `captures` table
 - Update `zone_ownership` table
 - Set `fresh_paint_until = now() + 30 minutes` (immunity period)
 - Trigger haptic feedback + sound + visual celebration on client
 - Push notification to previous owner (if any): "🔔 [Username] stole your hex in [Neighbourhood]"

To steal an owned hex (held by another user):

- Identical to capture, BUT:
- If `fresh_paint_until > now()` → CANNOT steal, show "🛑 Fresh Paint — try again at HH:MM"
- If `fortified_until > now()` → dwell time doubles to 45 seconds (Fortify is a power-up, post-MVP)
- Successful steal awards 1.3x IP (steal bonus to incentivise contested play)

Assist (multiple users in same hex simultaneously):

- All users in the hex start their own 20-second dwell timer independently
- First user to complete 20s captures the hex and gets full IP
- Other users who were in the hex for ≥ 10 seconds when capture happened get **Completion Bonus = 30% of IP** (no PPH, no ownership)
- This rewards group walks (friends walking together) without flooding ownership

Revisit (capturing your own hex again):

- Allowed only after `captured_at + 23 hours`

- Awards 50% of normal IP
- Resets `fresh_paint_until` and the held-since timestamp
- Useful for resetting decay (see 2.5) without losing the hex

Block / Cooldown:

- After any capture (yours or by anyone), the hex enters a 15-minute "block" period
- During block, NO ONE can capture it — including the person who just captured it
- Prevents rapid-fire trading between two players

2.3 Hex Hoarding Limits

- **Soft cap: 50 simultaneously held hexes.** Once you hold 50, you stop earning PPH on additional captures (you still own them, but no income)
- **Hard cap: 200 simultaneously held hexes.** Beyond 200, new captures fail with "Empire too big — release some hexes or hold fewer"
- **Release hex:** users can voluntarily release a hex from their profile to free up cap space

2.4 Points System (THE ECONOMY)

Two types of points:

A. Round Points (RP) — resets every calendar month at 5 AM IST on the 1st

- Used for monthly leaderboard rankings
- Earned via captures, steals, assists, daily streaks, medals
- This is the competitive number

B. Lifetime Points (LP) — persistent forever

- Used for level progression (see 2.6)
- All RP also count as LP (RP is a subset)
- Plus bonus LP from medal awards

Point sources and values:

Source	RP	LP	Notes
Capture neutral hex	+IP	+IP	IP from hex's <code>ip_value</code> (50-250)
Steal owned hex	+IP × 1.3	+IP × 1.3	Steal bonus
Revisit own hex	+IP × 0.5	+IP × 0.5	After 23-hr hold
Assist (in hex when captured by another)	+IP × 0.3	+IP × 0.3	Held ≥ 10 sec
Hold hex (PPH)	+PPH per hour	+PPH per hour	Up to soft cap of 50 hexes
Daily login	+20	+20	Resets at 5 AM IST
7-day streak	+100	+100	One-time per streak break
30-day streak	+500	+500	
100-day streak	+2000	+2000	
6,000+ steps in a day	+50	+50	Via HealthKit/Health Connect
First capture in new neighbourhood	+75	+75	Encourages exploration
Medal earned	+varies	+varies	See medal list (Phase 6)

Daily cap: 800 points/day max for unlevelled users. See levels (2.6) for higher caps.

2.5 Hex Decay (POST-MVP, Month 2)

To prevent dead empires, owned hexes decay if you never revisit them:

- Day 0-7 after capture: full PPH
- Day 8-14: 70% PPH
- Day 15-21: 40% PPH
- Day 22-28: 10% PPH
- Day 29: hex auto-released back to neutral
- Walking through the hex (no full capture needed) resets the decay clock — just being inside it for 10 seconds = "visited," decay restarts

MVP simplification: No decay in Phase 1-9. Decay added in Phase 10 alongside Paths.

2.6 Levels & Safe Points (StepSetGo-inspired)

5 levels, each with daily LP earning caps and minimum activity to stay:

Level	LP Range	Daily Cap (RP)	Safe Points (daily min to retain)	Unlocks
1 — Walker	0 - 500	200	none	Base game
2 — Strider	500 - 2,500	400	1 capture/day	Friend list
3 — Patroller	2,500 - 10,000	600	50 RP/day, 5-day streak required	Clans, custom username colour
4 — Conqueror	10,000 - 30,000	800	100 RP/day, 10-day streak	Clan creation, profile titles
5 — Mayor	30,000+	1,000	150 RP/day, 15-day streak	Custom hex naming, founder badge

Demotion: Miss your Safe Point requirement 3 consecutive days → drop 1 level.

- Exception: 1 "vacation pause" token per month (auto-issued)
- Demotion notification: "🚩 You've dropped to [Level]. Walk 100 RP today to recover."

Promotion: Cross LP threshold → instant level-up celebration screen + medal + +1000 RP bonus.

2.7 Seasons / Rounds

- **Calendar month** = one season
- Resets: 1st of each month at 5:00 AM IST
- On reset:
 - All RP zeros out
 - All hex ownerships persist (you keep your territory across seasons)
 - Monthly leaderboards finalised; top 10 in each pincode get a "Season X Champion" medal
 - PPH continues uninterrupted

2.8 Streaks

- A "streak day" = at least 1 successful hex capture between 5 AM today and 5 AM tomorrow IST
- Streak counter shown prominently on home screen with flame emoji
- Streak Freeze: auto-granted 1 per 30 days; saves your streak if you miss a day

- Push notification at 9 PM IST if no capture yet: "🔥 Your 12-day streak is at risk!"

2.9 Medals (Launch Set — 8 Medals)

Medal	Trigger	LP Reward	Display
First Blood	First capture ever	+50	Bronze badge
Week One	7-day streak	+100	Bronze badge
Month One	30-day streak	+500	Silver badge
Centurion	100 lifetime captures	+200	Silver badge
Insomniac	Capture between 12 AM – 5 AM IST	+50	Bronze badge
Monsoon Warrior	25 captures during active rainfall	+300	Silver badge
Neighbourhood King	Hold #1 spot in your pincode for 7 days	+500	Gold badge
Bridge Crosser	Capture hexes on both sides of any flyover within 30 min	+100	Bronze badge

Post-MVP medals (Phase 6+): Take-1000, Take-10000, Festival event medals (Diwali Dazzler, Pongal Conqueror, Onam Pookalam Pro), IT Park medals (Manyata Marauder, Bagmane Baron), college campus medals, water hex medal (Lake Lord), 365-day streak (Year of Walking), Crown Hex medals (one per pincode).

2.10 Anti-Cheat Rules Summary (full detail in Phase 8)

A capture is invalidated if ANY of these are true:

- GPS marked as `isMock = true`
- Sustained speed > 10 km/h during dwell
- Zero accelerometer step events during 20-second dwell
- GPS accuracy > 25 meters
- Impossible jump (>5 km in <90 seconds since last GPS ping)
- Play Integrity fails on Android, App Attest fails on iOS
- Risk score ≥ 80 (composite)

Three-strike escalation:

- Strike 1: Warning + 24-hour shadow-ban from leaderboard
- Strike 2: 7-day full ban
- Strike 3: Permaban + phone number blacklist

2.11 Travel Mode (auto, not a cheat)

- Sustained GPS speed > 10 km/h for 60 seconds → auto-enter Travel Mode
- Travel Mode: location pings drop from every 5s to every 60s (battery save)
- Captures are disabled during Travel Mode
- Auto-exits when speed drops < 5 km/h for 60 seconds
- Show friendly toast: "🚗 Travel Mode on — saving your battery"
- Push notification when re-entering walk mode: "🚶 You're walking again. Resume capturing?"

3. TECHNICAL ARCHITECTURE

3.1 High-Level System Diagram





3.2 Critical Architectural Decisions

Decision 1: Expo Bare workflow, NOT managed.

- Reason: managed workflow's background location is too restricted. We need `react-native-background-geolocation` (transistorsoft, paid ~\$250 one-time licence) for production-grade tracking
- Alternative for first 2 phases: `expo-location` + `expo-task-manager` (free, weaker, fine for dev). Migrate when battery issues appear

Decision 2: Supabase over Firebase.

- Postgres + PostGIS + h3-pg = native hex spatial queries
- Realtime subscriptions for "hex stolen" notifications
- Row Level Security gives clean multi-tenant safety
- \$25/month Pro tier from day 1 (free tier auto-pauses after 7 days inactive — production killer)

Decision 3: MapLibre over Google Maps.

- Google Maps SDK is hostile pricing at scale
- MapLibre is MIT-licensed; works with MapTiler (free 100K tiles/month) or self-hosted tiles
- Hex overlay performance: 10K hexes rendered smoothly on mid-range Android

Decision 4: H3 hex indices stored as bigint, not string.

- bigint comparisons are faster

- h3-pg works natively with bigint
- Save 50% space vs string storage

Decision 5: Capture validation happens BOTH client and server.

- Client: instant feedback (timer, animation), best-effort anti-cheat
- Server: source of truth, full anti-cheat suite, can reject capture post-hoc
- Conflict: server always wins; client must reconcile

3.3 Data Flow: Successful Capture

1. User taps "Capture" or auto-detected in hex
 2. Client starts dwell timer (20s)
 3. Client polls GPS every 1s during dwell:
 - Check still in hex (point-in-polygon via h3-js)
 - Check accuracy $\leq 25\text{m}$
 - Check speed $\leq 10\text{ km/h}$
 - Increment accelerometer step count
 4. Timer hits 20s with all checks passing:
 - Client shows "Capturing..." $\rightarrow$ success animation
 - Client sends POST /capture with full GPS trace + accelerometer data
 5. Edge function `validate\_capture` runs:
 - Check user hasn't been banned
 - Check hex isn't in block cooldown
 - Check hex isn't fresh-painted by someone else
 - Check anti-cheat risk score < 80
 - Check capture hasn't been duplicated
 6. If valid:
 - INSERT into captures
 - UPSERT into zone_ownership
 - UPDATE user RP, LP, daily_cap_remaining
 - Trigger medal_check_function
 - Trigger push_to_previous_owner via Realtime
 7. Client receives confirmation:
 - Updates local zone state
 - Plays celebration animation
 - If first capture today, increments streak
 8. Background: previous owner's app gets Realtime ping:
 - Shows in-app notification banner
 - Sends push notification if app backgrounded
-

4. DATABASE SCHEMA (COMPLETE)

sql

```

-- Enable extensions
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS h3;
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

-- ===== USERS =====
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  phone TEXT UNIQUE NOT NULL,
  username TEXT UNIQUE,
  display_name TEXT,
  avatar_url TEXT,
  hex_colour TEXT DEFAULT '#FF6F00', -- saffron default
  level INT DEFAULT 1 CHECK (level BETWEEN 1 AND 5),
  lifetime_points BIGINT DEFAULT 0,
  current_round_points INT DEFAULT 0,
  current_streak INT DEFAULT 0,
  longest_streak INT DEFAULT 0,
  last_capture_at TIMESTAMPTZ,
  streak_freezes_available INT DEFAULT 1,
  vacation_tokens_available INT DEFAULT 1,
  pincode TEXT,
  home_neighbourhood TEXT,
  language_pref TEXT DEFAULT 'en' CHECK (language_pref IN ('en', 'hi', 'kn', 'ta', 'te')),
  ghost_mode BOOLEAN DEFAULT FALSE,
  notification_prefs JSONB DEFAULT '{"steal":true,"streak":true,"daily":true}',
  daily_cap_remaining INT DEFAULT 200,
  daily_cap_reset_at TIMESTAMPTZ,
  clan_id UUID,
  current_held_hexes INT DEFAULT 0,
  strikes INT DEFAULT 0 CHECK (strikes BETWEEN 0 AND 3),
  banned_until TIMESTAMPTZ,
  banned_permanently BOOLEAN DEFAULT FALSE,
  device_fingerprint TEXT,
  flags JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX users_pincode_idx ON users(pincode) WHERE banned_permanently = FALSE;
CREATE INDEX users_round_points_idx ON users(current_round_points DESC);
CREATE INDEX users_clan_idx ON users(clan_id);

-- ===== HEXES =====
CREATE TABLE hexes (
  h3_index BIGINT PRIMARY KEY,
  centroid_lat FLOAT8 NOT NULL,

```

```

centroid_lng FLOAT8 NOT NULL,
boundary_geojson JSONB NOT NULL,
boundary GEOMETRY(Polygon, 4326) NOT NULL,
pincode TEXT,
neighbourhood TEXT,
rarity INT DEFAULT 1 CHECK (rarity BETWEEN 1 AND 5),
ip_value INT DEFAULT 100,
pph_value INT DEFAULT 3,
is_water BOOLEAN DEFAULT FALSE,
is_park BOOLEAN DEFAULT FALSE,
is_crown BOOLEAN DEFAULT FALSE,
poi_type TEXT,
is_power_zone BOOLEAN DEFAULT FALSE,
power_zone_sponsor TEXT,
created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX hexes_pincode_idx ON hexes(pincode);
CREATE INDEX hexes_neighbourhood_idx ON hexes(neighbourhood);
CREATE INDEX hexes_boundary_gix ON hexes USING GIST(boundary);
CREATE INDEX hexes_centroid_idx ON hexes(centroid_lat, centroid_lng);

-- ===== ZONE OWNERSHIP =====
CREATE TABLE zone_ownership (
  h3_index BIGINT PRIMARY KEY REFERENCES hexes(h3_index),
  user_id UUID REFERENCES users(id),
  clan_id UUID,
  captured_at TIMESTAMPTZ NOT NULL,
  fresh_paint_until TIMESTAMPTZ,
  fortified_until TIMESTAMPTZ,
  last_visited_at TIMESTAMPTZ,
  block_cooldown_until TIMESTAMPTZ,
  pph_paused BOOLEAN DEFAULT FALSE, -- TRUE if user has > 50 hexes (soft cap)
  updated_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX zone_ownership_user_idx ON zone_ownership(user_id);
CREATE INDEX zone_ownership_clan_idx ON zone_ownership(clan_id);

-- ===== CAPTURES (history log) =====
CREATE TABLE captures (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  h3_index BIGINT REFERENCES hexes(h3_index),
  captured_at TIMESTAMPTZ DEFAULT now(),
  previous_owner_id UUID REFERENCES users(id),
  capture_type TEXT CHECK (capture_type IN ('neutral', 'steal', 'revisit', 'assist')),
  ip_awarded INT NOT NULL,
  hold_duration_seconds INT,

```

```

gps_accuracy_m FLOAT,
gps_speed_kmh FLOAT,
accelerometer_steps INT,
device_integrity_score INT,
risk_score INT,
round_id TEXT,
is_validated BOOLEAN DEFAULT TRUE,
validation_notes TEXT
);

CREATE INDEX captures_user_idx ON captures(user_id, captured_at DESC);
CREATE INDEX captures_hex_idx ON captures(h3_index, captured_at DESC);
CREATE INDEX captures_round_idx ON captures(round_id, captured_at DESC);

-- ===== MEDALS =====
CREATE TABLE medals (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  description TEXT,
  icon_url TEXT,
  tier TEXT CHECK (tier IN ('bronze', 'silver', 'gold', 'platinum')),
  lp_reward INT DEFAULT 0,
  criteria_type TEXT,
  criteria_threshold INT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE user_medals (
  user_id UUID REFERENCES users(id),
  medal_id TEXT REFERENCES medals(id),
  earned_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, medal_id)
);

-- ===== FRIENDSHIPS =====
CREATE TABLE friendships (
  user_a UUID REFERENCES users(id),
  user_b UUID REFERENCES users(id),
  status TEXT CHECK (status IN ('pending', 'accepted', 'blocked')) DEFAULT 'pending',
  initiated_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_a, user_b),
  CHECK (user_a < user_b) -- always store lower UUID first
);

CREATE INDEX friendships_a_idx ON friendships(user_a, status);
CREATE INDEX friendships_b_idx ON friendships(user_b, status);

```

```
-- ===== CLANS (Phase 11) =====
```

```
CREATE TABLE clans (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name TEXT UNIQUE NOT NULL,  
  tag TEXT UNIQUE NOT NULL, -- 3-5 char prefix like [HSR7]  
  description TEXT,  
  type TEXT CHECK (type IN ('apartment', 'college', 'techpark', 'custom')),  
  pincode TEXT,  
  neighbourhood TEXT,  
  verified BOOLEAN DEFAULT FALSE,  
  created_by UUID REFERENCES users(id),  
  member_count INT DEFAULT 1,  
  total_round_points BIGINT DEFAULT 0,  
  total_held_hexes INT DEFAULT 0,  
  hex_colour TEXT,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

```
CREATE TABLE clan_members (  
  clan_id UUID REFERENCES clans(id),  
  user_id UUID REFERENCES users(id),  
  role TEXT CHECK (role IN ('member', 'officer', 'leader')) DEFAULT 'member',  
  joined_at TIMESTAMPTZ DEFAULT now(),  
  PRIMARY KEY (clan_id, user_id)  
);
```

```
-- ===== PATHS (Phase 10) =====
```

```
CREATE TABLE paths (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name TEXT NOT NULL,  
  description TEXT,  
  neighbourhood TEXT,  
  pincode TEXT,  
  distance_meters INT,  
  walking_time_cap_min INT,  
  running_time_cap_min INT,  
  ip_value INT,  
  pph_value INT,  
  checkpoints JSONB, -- array of {lat, lng, order, dwell_sec}  
  polyline_geojson JSONB,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

```
CREATE TABLE path_ownership (  
  path_id UUID PRIMARY KEY REFERENCES paths(id),  
  user_id UUID REFERENCES users(id),
```

```
    captured_at TIMESTAMPTZ,  
    best_completion_time_sec INT,  
    total_completions INT DEFAULT 0  
);
```

```
CREATE TABLE path_attempts (  
    id BIGSERIAL PRIMARY KEY,  
    user_id UUID REFERENCES users(id),  
    path_id UUID REFERENCES paths(id),  
    started_at TIMESTAMPTZ DEFAULT now(),  
    completed_at TIMESTAMPTZ,  
    completion_time_sec INT,  
    was_successful BOOLEAN,  
    checkpoint_log JSONB,  
    avg_speed_kmh FLOAT,  
    was_validated BOOLEAN DEFAULT TRUE,  
    ip_awarded INT,  
    failure_reason TEXT  
);
```

```
-- ===== ANTI-CHEAT LOG =====
```

```
CREATE TABLE anti_cheat_events (  
    id BIGSERIAL PRIMARY KEY,  
    user_id UUID REFERENCES users(id),  
    event_type TEXT,  
    capture_id BIGINT REFERENCES captures(id),  
    risk_score INT,  
    detection_flags JSONB,  
    action_taken TEXT,  
    reviewed BOOLEAN DEFAULT FALSE,  
    reviewer_notes TEXT,  
    created_at TIMESTAMPTZ DEFAULT now()  
);
```

```
CREATE INDEX ac_user_idx ON anti_cheat_events(user_id, created_at DESC);
```

```
CREATE INDEX ac_unreviewed_idx ON anti_cheat_events(created_at DESC) WHERE reviewed
```

```
-- ===== SESSIONS / GPS PINGS (for fraud review) =====
```

```
CREATE TABLE gps_pings (  
    id BIGSERIAL PRIMARY KEY,  
    user_id UUID REFERENCES users(id),  
    recorded_at TIMESTAMPTZ NOT NULL,  
    lat FLOAT8 NOT NULL,  
    lng FLOAT8 NOT NULL,  
    accuracy_m FLOAT,  
    speed_kmh FLOAT,  
    is_mock BOOLEAN DEFAULT FALSE,  
    session_id UUID
```

```

);
-- This table will get massive; partition by month for production
CREATE INDEX gps_pings_user_idx ON gps_pings(user_id, recorded_at DESC);

-- ===== NOTIFICATIONS QUEUE =====
CREATE TABLE notifications (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  type TEXT,
  title TEXT,
  body TEXT,
  data JSONB,
  delivered BOOLEAN DEFAULT FALSE,
  read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX notif_user_undeliv_idx ON notifications(user_id) WHERE delivered = FALSE;

-- ===== ROW LEVEL SECURITY =====
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
CREATE POLICY users_read_own ON users FOR SELECT USING (auth.uid() = id);
CREATE POLICY users_read_others_limited ON users FOR SELECT USING (TRUE);
CREATE POLICY users_update_own ON users FOR UPDATE USING (auth.uid() = id);

ALTER TABLE captures ENABLE ROW LEVEL SECURITY;
CREATE POLICY captures_read_all ON captures FOR SELECT USING (TRUE);
-- Inserts only via Edge Function with service role

ALTER TABLE zone_ownership ENABLE ROW LEVEL SECURITY;
CREATE POLICY zone_read_all ON zone_ownership FOR SELECT USING (TRUE);

-- (Add more RLS policies as needed for friendships, clans, etc.)

```

5. TECH STACK & DEPENDENCIES

5.1 Mobile (Expo Bare React Native)

```

json

```

```
{
  "name": "hexa",
  "version": "0.1.0",
  "main": "expo-router/entry",
  "scripts": {
    "start": "expo start --dev-client",
    "android": "expo run:android",
    "ios": "expo run:ios",
    "build:ios": "eas build --platform ios --profile preview",
    "build:android": "eas build --platform android --profile preview",
    "lint": "eslint . --ext .ts,.tsx",
    "test": "jest"
  },
  "dependencies": {
    "expo": "^51.0.0",
    "expo-router": "^3.5.0",
    "expo-location": "^17.0.0",
    "expo-task-manager": "^11.8.0",
    "expo-sensors": "^13.0.0",
    "expo-haptics": "^13.0.0",
    "expo-notifications": "^0.28.0",
    "expo-secure-store": "^13.0.0",
    "expo-image": "^1.12.0",
    "expo-blur": "^13.0.0",
    "expo-linear-gradient": "^13.0.0",
    "expo-status-bar": "~1.12.0",
    "expo-system-ui": "~3.0.0",
    "expo-localization": "~15.0.0",
    "expo-constants": "~16.0.0",
    "expo-application": "~5.9.0",
    "expo-device": "~6.0.0",
    "expo-clipboard": "~6.0.0",
    "expo-sharing": "~12.0.0",
    "expo-file-system": "~17.0.0",
    "expo-image-manipulator": "~12.0.0",
    "react": "18.2.0",
    "react-native": "0.74.5",
    "react-native-svg": "^15.2.0",
    "react-native-reanimated": "~3.10.0",
    "react-native-gesture-handler": "~2.16.0",
    "react-native-safe-area-context": "4.10.0",
    "react-native-screens": "~3.31.0",
    "@maplibre/maplibre-react-native": "^10.0.0",
    "@supabase/supabase-js": "^2.43.0",
    "h3-js": "^4.1.0",
    "zustand": "^4.5.0",
  }
}
```

```

    "@tanstack/react-query": "^5.40.0",
    "react-native-mmkv": "^2.12.0",
    "react-native-keyboard-controller": "^1.12.0",
    "react-native-toast-message": "^2.2.0",
    "date-fns": "^3.6.0",
    "viem": "^2.10.0"
  },
  "devDependencies": {
    "@babel/core": "^7.24.0",
    "@types/react": "~18.2.0",
    "typescript": "~5.3.0",
    "eslint": "^8.57.0",
    "eslint-config-expo": "^7.1.0",
    "prettier": "^3.2.0",
    "jest": "^29.7.0",
    "jest-expo": "~51.0.0"
  }
}

```

Note on `react-native-background-geolocation` (transistorsoft):

- Add this in Phase 3 when background GPS becomes critical
- ~\$250 one-time licence; needed for production-grade tracking
- For Phases 1-2, use only `expo-location` + `expo-task-manager`

5.2 Backend (Supabase)

Plan: Supabase Pro (\$25/mo) from day 1

Extensions to enable:

- PostGIS
- h3-pg
- uuid-oss
- pg_cron (for scheduled jobs)
- pg_net (for HTTP calls from DB)

Edge Functions (Deno + TypeScript):

- `validate_capture` — POST endpoint, runs full anti-cheat + writes to DB
- `daily_reset` — pg_cron job at 5 AM IST, resets daily caps + streak checks
- `monthly_reset` — pg_cron job 1st of month 5 AM IST, season rollover
- `medal_check` — Postgres trigger after capture, evaluates medal eligibility
- `send_notification` — Wraps Expo Push API
- `leaderboard_calc` — Materialized view refresh, runs every 5 min

5.3 External Services

Service	Purpose	Cost (initial)
Supabase Pro	DB + Auth + Realtime + Storage	\$25/mo
MapTiler	Map tiles	Free (100K tiles/mo)
MSG91	OTP SMS in India	₹0.18 per SMS
Expo EAS Build	iOS/Android builds	\$19/mo Production
Apple Developer	App Store	\$99/yr
Google Play	Play Store	\$25 one-time
Sentry	Error tracking	Free (5K events/mo)
PostHog	Analytics	Free (1M events/mo)
OpenWeather	Monsoon mode detection	Free (1000 calls/day)

Total monthly burn (pre-revenue, dev phase): ~\$50-60/month ≈ ₹4,500/mo **Annual cost (Apple + Play):** ~₹10,000

5.4 Development Tools

- **IDE:** Cursor or VS Code with Claude Code extension
- **Version control:** GitHub (private repo)
- **Design:** Figma free tier
- **API testing:** Postman or Insomnia
- **DB GUI:** TablePlus or Supabase Studio (built-in)
- **Mobile debugging:** Expo Dev Tools + Flipper

6. PROJECT STRUCTURE

```
hexa/
├─ app/                                # Expo Router file-based routing
│   ├── _layout.tsx                   # Root layout, auth guards
│   ├── (auth)/                       # Auth stack
│   │   ├── phone.tsx
│   │   ├── otp.tsx
│   │   └─ profile-setup.tsx
```

```
|   |─ (tabs)/                                # Main tabs after auth
|   |   |─ _layout.tsx
|   |   |─ index.tsx                        # Map (home)
|   |   |─ leaderboard.tsx
|   |   |─ friends.tsx
|   |   |─ profile.tsx
|   |─ capture/[hexId].tsx                  # Capture flow modal
|   |─ hex/[hexId].tsx                     # Hex detail
|   |─ user/[userId].tsx                   # User profile
|   |─ medal/[medalId].tsx                 # Medal detail
|   |─ settings.tsx
|
|─ components/                             # Reusable UI
|   |─ map/
|   |   |─ HexaMap.tsx                     # MapLibre wrapper
|   |   |─ HexLayer.tsx                   # Hex polygon rendering
|   |   |─ UserMarker.tsx
|   |   |─ CaptureOverlay.tsx
|   |─ capture/
|   |   |─ DwellTimer.tsx                 # 20-second progress ring
|   |   |─ CaptureSuccess.tsx
|   |   |─ ShareCard.tsx
|   |─ ui/
|   |   |─ Button.tsx
|   |   |─ Card.tsx
|   |   |─ Toast.tsx
|   |   |─ Badge.tsx
|   |   |─ Avatar.tsx
|   |─ shared/
|   |   |─ StreakIndicator.tsx
|   |   |─ LevelBadge.tsx
|
|─ lib/                                    # Pure logic, no UI
|   |─ h3/
|   |   |─ hexUtils.ts                    # Wrappers around h3-js
|   |   |─ pointInHex.ts
|   |   |─ neighbouringHexes.ts
|   |─ capture/
|   |   |─ captureFSM.ts                  # Finite state machine
|   |   |─ dwellTimer.ts
|   |   |─ validators.ts
|   |─ location/
|   |   |─ backgroundTask.ts
|   |   |─ locationStore.ts
|   |   |─ travelMode.ts
|   |─ antiCheat/
|   |   |─ speedCheck.ts
```

```
| | |─ mockDetection.ts
| | |─ accelerometerCorrelation.ts
| | └─ riskScorer.ts
| └─ points/
|   |─ pointsCalculator.ts
|   |─ levelManager.ts
|   └─ safePoints.ts
| └─ medals/
|   |─ medalDefinitions.ts
|   └─ medalEvaluator.ts
| └─ supabase/
|   |─ client.ts
|   |─ auth.ts
|   └─ queries.ts
| └─ utils/
|   |─ dates.ts          # IST timezone helpers
|   |─ formatters.ts
|   └─ constants.ts
|
|─ stores/                # Zustand
|   |─ userStore.ts
|   |─ mapStore.ts
|   |─ captureStore.ts
|   └─ notificationStore.ts
|
|─ hooks/
|   |─ useLocation.ts
|   |─ useNearbyHexes.ts
|   |─ useCapture.ts
|   |─ useCurrentUser.ts
|   |─ useStreak.ts
|   └─ useRealtime.ts
|
|─ types/
|   |─ database.ts        # Generated from Supabase
|   |─ game.ts            # Hex, Capture, etc.
|   └─ api.ts
|
|─ assets/
|   |─ images/
|   |─ sounds/
|   |─ fonts/
|   └─ medals/            # SVG medal icons
|
|─ scripts/                # One-off scripts
|   |─ generate-hexes.ts  # Pre-seed hexes
|   └─ seed-medals.ts
```

```
|   └─ seed-test-users.ts
|
└─ supabase/                                # Backend
    └─ migrations/
        ├── 001_init.sql
        ├── 002_hexes.sql
        ├── 003_captures.sql
        └─ ...
    └─ functions/                          # Edge Functions
        ├── validate-capture/
            └─ index.ts
        ├── daily-reset/
        ├── monthly-reset/
        ├── medal-check/
        └─ leaderboard-calc/
    └─ seed.sql
|
└─ app.json                                # Expo config
└─ eas.json                                # EAS build config
└─ package.json
└─ tsconfig.json
└─ babel.config.js
└─ .env.local                              # Local env vars (gitignored)
```

PHASE 0 — Pre-Build Setup (Day 0-3)

Goal: Accounts, repos, decisions locked. No code yet.

Tasks

1. Create accounts:

- GitHub (private repo **hexa**)
- Expo account
- Supabase (create new project, note URL + anon key + service_role key)
- MapTiler (get API key, note free tier 100K/mo)
- MSG91 (note OTP API key + Sender ID)
- Apple Developer (\$99/yr) — start enrollment now, takes 2-7 days
- Google Play Console (\$25 one-time)
- Sentry (free org)
- PostHog (free org)
- OpenWeather (free API key)

2. Lock in product decisions:

- App name: **Hexa** (confirm domain `hexa.app` or `playhexa.com` available)
- Reserve Instagram/X/TikTok handles: `@hexa.app`, `@hexaplay`, etc.
- Brand colour: saffron `#FF6F00`, dark `#1A1A1A`, accent green `#2E7D32`
- Tagline: "Walk the city. Capture it."

3. Create Figma file:

- Wireframe 8 core screens: Phone OTP, Profile Setup, Map (home), Capture (modal), Hex Detail, Profile, Leaderboard, Share Card
- Don't polish — rough wireframes only
- Time budget: 1 day

4. Initialize repo:

bash

```
npx create-expo-app hexa --template tabs@52
cd hexa
git init
git remote add origin git@github.com:USERNAME/hexa.git
git push -u origin main
```

5. Run Expo prebuild to enter Bare workflow:

bash

```
npx expo prebuild --clean
```

6. Set up `.env.local`:

```
EXPO_PUBLIC_SUPABASE_URL=
EXPO_PUBLIC_SUPABASE_ANON_KEY=
EXPO_PUBLIC_MAPTILER_KEY=
EXPO_PUBLIC_POSTHOG_KEY=
EXPO_PUBLIC_SENTRY_DSN=
SUPABASE_SERVICE_ROLE_KEY= # Server-only, NEVER bundle
MSG91_AUTH_KEY= # Server-only
```

Acceptance Criteria

- ☐ All accounts created and credentials saved in 1Password / Bitwarden
- ☐ Empty Expo project boots on iOS simulator + Android emulator
- ☐ Supabase project responds to a `select 1` query
- ☐ Figma file has 8 wireframes (rough is fine)
- ☐ `.env.local` populated, `.env.local` in `.gitignore`

PHASE 1 — Foundation & Auth (Week 1)

Goal: User can sign up with phone OTP, set username + pincode, see an empty home screen.

Tasks

1.1 Supabase schema — first migration

Create `supabase/migrations/001_init.sql` with:

- `users` table (full schema from Section 4)
- Enable extensions: postgis, h3, uuid-oss
- RLS policies for users

Run via Supabase CLI:

```
bash
npx supabase migration up
```

1.2 Auth flow

Screens:

1. `app/(auth)/phone.tsx` — input phone (+91 prefix locked), validates 10 digits
2. `app/(auth)/otp.tsx` — 6-digit OTP input, auto-detect from SMS
3. `app/(auth)/profile-setup.tsx` — username + pincode + display name + language

Logic:

- Use Supabase Auth with phone provider
- Supabase routes OTP via Twilio by default — switch to MSG91 via custom Edge Function for India pricing
- On successful OTP, check if `users` row exists for that phone; if not, redirect to profile-setup
- Profile setup writes `users` row with `phone`, `username` (validate unique + alphanumeric 3-20 chars), `pincode` (validate Bangalore range 560001-560103), `language_pref`

Edge Function `send_otp`:

```
typescript
```

```
// supabase/functions/send-otp/index.ts
import { serve } from 'std/server'

serve(async (req) => {
  const { phone } = await req.json()
  // Validate +91 prefix and 10-digit number
  if (!/^\+91[6-9]\d{9}$/.test(phone)) {
    return new Response(JSON.stringify({error: 'invalid_phone'}), {status: 400})
  }

  const otp = Math.floor(100000 + Math.random() * 900000).toString()

  // Store hashed OTP in temp table with 5-min expiry
  // Send via MSG91 API
  const msg91Response = await fetch('https://api.msg91.com/api/v5/otp', {
    method: 'POST',
    headers: {
      'authkey': Deno.env.get('MSG91_AUTH_KEY')!,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      template_id: Deno.env.get('MSG91_TEMPLATE_ID')!,
      mobile: phone.replace('+', ''),
      otp,
      otp_length: 6,
      otp_expiry: 5
    })
  })

  return new Response(JSON.stringify({success: true}))
})
```

1.3 Auth state management

stores/userStore.ts:

typescript

```

import { create } from 'zustand'
import { persist, createJSONStorage } from 'zustand/middleware'
import { MMKV } from 'react-native-mmkv'

const storage = new MMKV()

interface UserStore {
  user: User | null
  isLoading: boolean
  setUser: (user: User | null) => void
  signOut: () => Promise<void>
}

export const useUserStore = create<UserStore>()(
  persist(
    (set) => ({
      user: null,
      isLoading: false,
      setUser: (user) => set({user}),
      signOut: async () => {
        await supabase.auth.signOut()
        set({user: null})
      }
    }),
    {
      name: 'user-store',
      storage: createJSONStorage(() => ({
        getItem: (name) => storage.getString(name) ?? null,
        setItem: (name, value) => storage.set(name, value),
        removeItem: (name) => storage.delete(name)
      })))
  )
)

```

1.4 Root layout with auth guard

`app/_layout.tsx`:

- Loads user on mount via `supabase.auth.getSession()`
- Redirects to `/auth/phone` if no session
- Redirects to `/tabs` if authenticated and profile complete
- Redirects to `/auth/profile-setup` if authenticated but profile incomplete

1.5 Home screen placeholder

- Render simple "Welcome [username]" + "Map coming soon" text
- Bottom tab nav with 4 tabs: Map, Leaderboard, Friends, Profile

Acceptance Criteria

- ☐ Phone OTP flow works end-to-end on iOS + Android
- ☐ User row created in DB after profile setup
- ☐ Auth state persists across app restarts (via MMKV)
- ☐ Sign out works and clears state
- ☐ Bottom tabs visible after auth
- ☐ All errors logged to Sentry

What NOT to do in Phase 1

- ❌ Don't build the map yet
 - ❌ Don't add OAuth/Google/Apple sign-in (phone only for MVP)
 - ❌ Don't add multi-language UI yet (English only; `language_pref` stored but not applied)
 - ❌ Don't optimize OTP UI animations
-

PHASE 2 — Map & Hex Display (Week 2)

Goal: Render Bangalore map with hex overlay. No interactions yet.

Tasks

2.1 MapLibre setup

Install:

```
bash

npm install @maplibre/maplibre-react-native
cd ios && pod install
```

iOS: Add to `Info.plist`:

```
xml

<key>NSLocationWhenInUseUsageDescription</key>
<string>Hexa needs your location to capture hexes near you.</string>
<key>NSLocationAlwaysAndWhenInUseUsageDescription</key>
<string>Hexa tracks your walks in the background to defend your hexes.</string>
```

Android: Add to `AndroidManifest.xml`:

xml

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_LOCATION"/>
```

2.2 HexaMap component

components/map/HexaMap.tsx:

- Initialize MapLibre with MapTiler style URL (use light theme initially)
- Default centre: 12.9351°N, 77.6244°E (HSR Layout)
- Default zoom: 16 (street level)
- Disable tilt, pitch, compass (keep it simple)
- Show user location dot via `<UserLocation />`

2.3 Hex rendering layer

The challenge: don't render all 500K Bangalore hexes at once. Render only hexes in current viewport.

components/map/HexLayer.tsx:

typescript

```

import { useQuery } from '@tanstack/react-query'
import { ShapeSource, FillLayer, LineLayer } from '@maplibre/maplibre-react-native'

export function HexLayer({ viewportBounds }: { viewportBounds: BoundingBox }) {
  const { data: hexes } = useQuery({
    queryKey: ['hexes', viewportBounds],
    queryFn: () => fetchHexesInBounds(viewportBounds),
    enabled: !!viewportBounds,
    staleTime: 30000
  })

  const featureCollection = useMemo(() => ({
    type: 'FeatureCollection',
    features: (hexes ?? []).map(h => ({
      type: 'Feature',
      geometry: h.boundary_geojson,
      properties: {
        h3_index: h.h3_index.toString(),
        owner_colour: h.owner_colour ?? null,
        is_owned: !!h.owner_id,
        is_self: h.owner_id === currentUserId,
        rarity: h.rarity
      }
    }
  )))
  }, [hexes, currentUserId])

  return (
    <ShapeSource id="hexes" shape={featureCollection}>
      <FillLayer
        id="hex-fills"
        style={{
          fillColor: [
            'case',
            ['get', 'is_self'], '#FF6F00',
            ['get', 'is_owned'], ['get', 'owner_colour'],
            'rgba(200,200,200,0.2)'
          ],
          fillOpacity: 0.4
        }}
      />
      <LineLayer
        id="hex-outlines"
        style={{
          lineColor: '#666',
          lineWidth: 0.5,
          lineOpacity: 0.6
        }}
      />
    </ShapeSource>
  )
}

```

```

    }}
  />
</ShapeSource>
)
}

```

2.4 Fetch hexes in viewport

Supabase RPC function:

```

sql

CREATE OR REPLACE FUNCTION hexes_in_bounds(
  min_lat FLOAT8, min_lng FLOAT8,
  max_lat FLOAT8, max_lng FLOAT8
)
RETURNS TABLE (
  h3_index BIGINT,
  centroid_lat FLOAT8,
  centroid_lng FLOAT8,
  boundary_geojson JSONB,
  ip_value INT,
  pph_value INT,
  rarity INT,
  owner_id UUID,
  owner_username TEXT,
  owner_colour TEXT,
  fresh_paint_until TIMESTAMPTZ
) LANGUAGE sql AS $$
SELECT
  h.h3_index, h.centroid_lat, h.centroid_lng, h.boundary_geojson,
  h.ip_value, h.pph_value, h.rarity,
  zo.user_id AS owner_id,
  u.username AS owner_username,
  u.hex_colour AS owner_colour,
  zo.fresh_paint_until
FROM hexes h
LEFT JOIN zone_ownership zo ON zo.h3_index = h.h3_index
LEFT JOIN users u ON u.id = zo.user_id
WHERE h.boundary && ST_MakeEnvelope(min_lng, min_lat, max_lng, max_lat, 4326)
LIMIT 500;
$$;

```

Client side:

```

typescript

```

```
async function fetchHexesInBounds(bounds: BoundingBox) {
  const { data, error } = await supabase.rpc('hexes_in_bounds', {
    min_lat: bounds.south, min_lng: bounds.west,
    max_lat: bounds.north, max_lng: bounds.east
  })
  if (error) throw error
  return data
}
```

2.5 Map UI overlays

- Top bar: user avatar + current points
- Bottom info card (collapsed): "Tap a hex to see details"
- Floating action button: "Find nearest hex" (animates camera to nearest unowned hex)

Acceptance Criteria

- ☐ Map loads centred on HSR Layout
- ☐ User location dot appears (location permission granted)
- ☐ Hexes render as polygons in current viewport
- ☐ Pan/zoom triggers refetch of viewport hexes
- ☐ Owned hexes show in owner's colour, neutral in light grey
- ☐ Rendering stays at 60fps on mid-range Android (Pixel 6a or Realme equivalent)

What NOT to do

- ❌ Don't implement capture yet (Phase 4)
- ❌ Don't render hexes outside viewport
- ❌ Don't load all 500K hexes client-side ever

PHASE 3 — GPS, Location & Hex Generation (Week 3)

Goal: Background GPS tracking + pre-generate Bangalore hex grid.

Tasks

3.1 Run hex generation script (one-time backend operation)

`scripts/generate-hexes.ts` (full code in Appendix B):

This runs once, takes ~30 min, generates all H3 res-10 hexes for Bangalore bounding box, computes their boundaries, infers pincode + neighbourhood via reverse geocoding, and bulk-inserts into the `hexes` table.

Bangalore bounding box:

- North: 13.1737°N
- South: 12.8340°N
- East: 77.7798°E
- West: 77.4601°E
- Estimated hex count at res 10: ~520,000

Run:

```
bash  
  
npx tsx scripts/generate-hexes.ts
```

3.2 Pincode + neighbourhood assignment

Use Nominatim (free OpenStreetMap geocoder) batched at 1 req/sec to assign pincode + neighbourhood. For 520K hexes that's 6 days — too slow. Instead:

Alternative: Use Bangalore pincode polygon dataset (open data on github.com/datameet/india-pincodes), do PostGIS spatial join in batch:

```
sql  
  
UPDATE hexes h  
SET pincode = p.pincode, neighbourhood = p.neighbourhood  
FROM pincode_polygons p  
WHERE ST_Contains(p.boundary, ST_SetSRID(ST_MakePoint(h.centroid_lng, h.centroid_lat), 4326))
```

Runs in ~10 min on Supabase.

3.3 Rarity assignment

Assign rarity based on POI density:

```
sql
```

```

-- Mark park hexes
UPDATE hexes h SET is_park = TRUE, rarity = 3, ip_value = 170, pph_value = 5
WHERE EXISTS (
    SELECT 1 FROM osm_parks p
    WHERE ST_Intersects(p.boundary, h.boundary)
);

-- Mark water hexes
UPDATE hexes h SET is_water = TRUE, rarity = 4, ip_value = 210, pph_value = 7
WHERE EXISTS (
    SELECT 1 FROM osm_water_bodies w
    WHERE ST_Intersects(w.boundary, h.boundary)
);

-- Mark Crown hexes (1 per pincode – geographic centroid)
WITH centroids AS (
    SELECT pincode, ST_Centroid(ST_Collect(boundary)) AS centre
    FROM hexes GROUP BY pincode
),
crown_hexes AS (
    SELECT DISTINCT ON (c.pincode) h.h3_index
    FROM centroids c
    JOIN hexes h ON h.pincode = c.pincode
    ORDER BY c.pincode, ST_Distance(h.boundary, c.centre)
)
UPDATE hexes SET is_crown = TRUE, rarity = 5, ip_value = 250, pph_value = 9
WHERE h3_index IN (SELECT h3_index FROM crown_hexes);

```

3.4 Foreground GPS tracking

`lib/location/locationStore.ts`:

typescript

```

import * as Location from 'expo-location'
import { create } from 'zustand'

export const useLocationStore = create<LocationState>((set) => ({
  currentLocation: null,
  isTracking: false,

  startForegroundTracking: async () => {
    const { status } = await Location.requestForegroundPermissionsAsync()
    if (status !== 'granted') throw new Error('permission_denied')

    const subscription = await Location.watchPositionAsync({
      accuracy: Location.Accuracy.BestForNavigation,
      TimeInterval: 1000,
      distanceInterval: 1
    }, (location) => {
      set({currentLocation: location})
    })

    set({isTracking: true, _subscription: subscription})
  },

  stopForegroundTracking: () => {
    get()._subscription?.remove()
    set({isTracking: false, currentLocation: null})
  }
}))

```

3.5 Background GPS tracking (with expo-task-manager)

`lib/location/backgroundTask.ts`:

typescript

```
import * as TaskManager from 'expo-task-manager'
import * as Location from 'expo-location'

const BG_LOCATION_TASK = 'hexa-bg-location'

TaskManager.defineTask(BG_LOCATION_TASK, async ({data, error}) => {
  if (error) {
    Sentry.captureException(error)
    return
  }

  const { locations } = data as { locations: Location.LocationObject[] }
  const latest = locations[locations.length - 1]

  // Apply Travel Mode logic
  if (latest.coords.speed && latest.coords.speed * 3.6 > 10) {
    // Speed > 10 km/h, we're in Travel Mode
    // Persist GPS ping but skip capture checks
    await persistGpsPing(latest, { travelMode: true })
    return
  }

  // Check if user is inside any unowned hex
  const userHexId = latLngToH3(latest.coords.latitude, latest.coords.longitude, 10)
  const hexState = await getHexState(userHexId)

  if (hexState && !hexState.ownedByCurrentUser && !hexState.inBlockCooldown) {
    // Notify foreground UI via state
    triggerCaptureOpportunity(userHexId)
  }

  await persistGpsPing(latest, { travelMode: false })
})

export async function startBackgroundTracking() {
  const { status: fg } = await Location.requestForegroundPermissionsAsync()
  if (fg !== 'granted') return false

  const { status: bg } = await Location.requestBackgroundPermissionsAsync()
  if (bg !== 'granted') return false

  await Location.startLocationUpdatesAsync(BG_LOCATION_TASK, {
    accuracy: Location.Accuracy.Balanced,
    interval: 30000, // 30 sec in background to save battery
    distanceInterval: 50,
    foregroundService: {
```

```

        notificationTitle: 'Hexa is tracking your walk',
        notificationBody: 'Tap to open and capture hexes',
        notificationColor: '#FF6F00'
    },
    pausesUpdatesAutomatically: false,
    activityType: Location.ActivityType.Fitness
})

return true
}

```

3.6 Permissions dance UI

- First-launch onboarding: "Hexa needs your location to know which hexes you're in"
- Foreground permission first, then on first walk session, ask for background
- iOS: progressive permission (always-allow is hidden until "while using" granted first)
- Graceful degradation: if background denied, app still works in foreground

Acceptance Criteria

- ☐ All ~520K Bangalore hexes generated and stored
- ☐ All hexes have pincode + neighbourhood + rarity assigned
- ☐ Foreground GPS streams location updates every 1s when app open
- ☐ Background GPS works on iOS + Android (test: walk 500m with app backgrounded, check logs)
- ☐ Foreground service notification visible on Android during tracking
- ☐ Permission denial handled gracefully (no crash, clear message)
- ☐ GPS pings persisted to `gps_pings` table

What NOT to do

- **✗** Don't try to fix battery drain yet (Phase 8 optimisation)
- **✗** Don't switch to react-native-background-geolocation yet (Phase 8+)
- **✗** Don't request all permissions on app open — progressive flow

PHASE 4 — Capture Mechanics (Week 4-5)

Goal: User can capture, steal, assist, revisit hexes. Full game loop works.

Tasks

4.1 Capture Finite State Machine

`lib/capture/captureFSM.ts`:

States:

- `IDLE` — not in any hex
- `IN_HEX_NEUTRAL` — in an unowned hex, capture available
- `IN_HEX_OWN` — in own hex
- `IN_HEX_HOSTILE` — in enemy hex (steal available)
- `IN_HEX_FRESH_PAINT` — in hex with active immunity
- `IN_HEX_COOLDOWN` — in hex during block cooldown
- `DWELLING` — actively in 20-second capture timer
- `CAPTURING` — server validation in flight
- `CAPTURED` — success state, brief celebration
- `FAILED` — capture failed, show reason

Transitions defined as a state machine. Use `@xstate/react` if comfortable, or a simple switch-case if not.

4.2 Dwell Timer UI

`components/capture/DwellTimer.tsx`:

- Full-screen overlay with animated progress ring (Reanimated)
- Shows remaining seconds in big number
- Shows live GPS accuracy + speed (small)
- "Cancel" button stops the capture
- Auto-cancels if user leaves hex (reset to 0) or speed exceeds 10 km/h (pause timer)

typescript

```

function DwellTimer({ hexId, onSuccess, onFail }: Props) {
  const [secondsRemaining, setSecondsRemaining] = useState(20)
  const [stepCount, setStepCount] = useState(0)
  const { currentLocation } = useLocationStore()
  const hex = useHex(hexId)

  // Subscribe to accelerometer
  useEffect(() => {
    const sub = Accelerometer.addListener(detectStep(setStepCount))
    return () => sub.remove()
  }, [])

  // Tick timer
  useEffect(() => {
    if (!currentLocation) return

    // Check still in hex
    if (!isPointInHex(currentLocation.coords, hex.boundary)) {
      setSecondsRemaining(20) // reset
      return
    }

    // Check accuracy
    if (currentLocation.coords.accuracy > 25) {
      // Pause but don't reset
      return
    }

    // Check speed
    if (currentLocation.coords.speed * 3.6 > 10) {
      // Pause (don't reset)
      return
    }

    const t = setTimeout(() => setSecondsRemaining(s => s - 1), 1000)
    return () => clearTimeout(t)
  }, [currentLocation, secondsRemaining])

  // Complete
  useEffect(() => {
    if (secondsRemaining <= 0 && stepCount >= 2) {
      submitCapture()
    }
  }, [secondsRemaining, stepCount])

  return (

```

```
<CaptureOverlay
  progress={{(20 - secondsRemaining) / 20}}
  gps={currentLocation}
  stepCount={stepCount}
/>
)
}
```

4.3 Server-side capture validation

supabase/functions/validate-capture/index.ts:

typescript

```

import { serve } from 'std/server'
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  Deno.env.get('SUPABASE_URL')!,
  Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
)

interface CaptureRequest {
  h3_index: string // bigint serialized
  gps_trace: Array<{lat:number, lng:number, accuracy:number, speed:number, timestamp:number}>
  accelerometer_steps: number
  device_integrity_token?: string // Play Integrity / App Attest
  is_mock: boolean
}

serve(async (req) => {
  const user_id = (await supabase.auth.getUser(req.headers.get('Authorization'))).data.user.id
  const body: CaptureRequest = await req.json()

  // 1. Check user not banned
  const { data: user } = await supabase.from('users')
    .select('banned_until, banned_permanently, strikes, daily_cap_remaining, current_daily_cap')
    .eq('id', user_id).single()

  if (user.banned_permanently) return new Response(JSON.stringify({error: 'banned'}), {status: 403})
  if (user.banned_until && new Date(user.banned_until) > new Date()) {
    return new Response(JSON.stringify({error: 'banned_until', until: user.banned_until}), {status: 403})
  }

  // 2. Check daily cap
  if (user.daily_cap_remaining <= 0) {
    return new Response(JSON.stringify({error: 'daily_cap_reached'}), {status: 429})
  }

  // 3. Check hex state
  const { data: hex } = await supabase.from('hexes')
    .select('*').eq('h3_index', body.h3_index).single()
  if (!hex) return new Response(JSON.stringify({error: 'hex_not_found'}), {status: 404})

  const { data: ownership } = await supabase.from('zone_ownership')
    .select('*').eq('h3_index', body.h3_index).maybeSingle()

  // 4. Block cooldown check
  if (ownership?.block_cooldown_until && new Date(ownership.block_cooldown_until) > new Date()) {
    return new Response(JSON.stringify({error: 'block_cooldown', until: ownership.block_cooldown_until}), {status: 429})
  }
})

```

```

}

// 5. Fresh paint check (someone else's, not yours)
if (ownership && ownership.user_id !== user_id &&
    ownership.fresh_paint_until && new Date(ownership.fresh_paint_until) > new Date()) {
    return new Response(JSON.stringify({error: 'fresh_paint', until: ownership.fresh_paint_until}));
}

// 6. Hard cap check
if (user.current_held_hexes >= 200 && (!ownership || ownership.user_id !== user_id)) {
    return new Response(JSON.stringify({error: 'hex_cap_reached'}), {status: 429});
}

// 7. Anti-cheat
const riskScore = await calculateRiskScore({
    user_id,
    gps_trace: body.gps_trace,
    accelerometer_steps: body.accelerometer_steps,
    is_mock: body.is_mock,
    integrity_token: body.device_integrity_token,
    hex
});

if (riskScore >= 80) {
    // Log + strike
    await logAntiCheatEvent(user_id, 'auto_reject', riskScore);
    await incrementStrike(user_id);
    return new Response(JSON.stringify({error: 'capture_rejected', reason: 'integrity_token'}), {status: 429});
}

// 8. Determine capture type
const captureType =
    !ownership ? 'neutral' :
    ownership.user_id === user_id ? 'revisit' :
    'steal'

// 9. Calculate IP
let ip = hex.ip_value
if (captureType === 'steal') ip = Math.round(ip * 1.3)
if (captureType === 'revisit') ip = Math.round(ip * 0.5)

// Apply daily cap
ip = Math.min(ip, user.daily_cap_remaining)

// 10. Apply soft cap (PPH)
const pphPaused = user.current_held_hexes >= 50 && captureType !== 'revisit'

```

```

// 11. Write transaction
const { error: txErr } = await supabase.rpc('execute_capture', {
  p_user_id: user_id,
  p_h3_index: body.h3_index,
  p_capture_type: captureType,
  p_ip_awarded: ip,
  p_pph_paused: pphPaused,
  p_previous_owner: ownership?.user_id ?? null,
  p_gps_accuracy: body.gps_trace[body.gps_trace.length - 1].accuracy,
  p_gps_speed: body.gps_trace[body.gps_trace.length - 1].speed,
  p_accel_steps: body.accelerometer_steps,
  p_risk_score: riskScore
})

if (txErr) return new Response(JSON.stringify({error: 'tx_failed', detail: txErr.m

// 12. Notify previous owner via Realtime
if (ownership && ownership.user_id !== user_id) {
  await notifyHexStolen(ownership.user_id, user_id, body.h3_index, hex.neighbourho
}

// 13. Trigger medal check (async)
await supabase.functions.invoke('medal-check', { body: { user_id } })

return new Response(JSON.stringify({success: true, ip_awarded: ip, capture_type: c
})

```

4.4 `execute_capture` Postgres function

sql

```

CREATE OR REPLACE FUNCTION execute_capture(
    p_user_id UUID,
    p_h3_index BIGINT,
    p_capture_type TEXT,
    p_ip_awarded INT,
    p_pph_paused BOOLEAN,
    p_previous_owner UUID,
    p_gps_accuracy FLOAT,
    p_gps_speed FLOAT,
    p_accel_steps INT,
    p_risk_score INT
) RETURNS VOID LANGUAGE plpgsql AS $$
DECLARE
    v_round_id TEXT;
BEGIN
    -- Determine current round (YYYY-MM)
    v_round_id := to_char(now() AT TIME ZONE 'Asia/Kolkata', 'YYYY-MM');

    -- Insert capture record
    INSERT INTO captures (
        user_id, h3_index, captured_at, previous_owner_id,
        capture_type, ip_awarded, gps_accuracy_m, gps_speed_kmh,
        accelerometer_steps, risk_score, round_id
    ) VALUES (
        p_user_id, p_h3_index, now(), p_previous_owner,
        p_capture_type, p_ip_awarded, p_gps_accuracy, p_gps_speed,
        p_accel_steps, p_risk_score, v_round_id
    );

    -- Upsert ownership
    INSERT INTO zone_ownership (
        h3_index, user_id, captured_at,
        fresh_paint_until, last_visited_at, block_cooldown_until, pph_paused
    ) VALUES (
        p_h3_index, p_user_id, now(),
        now() + interval '30 minutes', now(),
        now() + interval '15 minutes', p_pph_paused
    ) ON CONFLICT (h3_index) DO UPDATE SET
        user_id = EXCLUDED.user_id,
        captured_at = EXCLUDED.captured_at,
        fresh_paint_until = EXCLUDED.fresh_paint_until,
        last_visited_at = EXCLUDED.last_visited_at,
        block_cooldown_until = EXCLUDED.block_cooldown_until,
        pph_paused = EXCLUDED.pph_paused,
        updated_at = now();

```

```

-- Update user points + cap
UPDATE users SET
  lifetime_points = lifetime_points + p_ip_awarded,
  current_round_points = current_round_points + p_ip_awarded,
  daily_cap_remaining = daily_cap_remaining - p_ip_awarded,
  last_capture_at = now(),
  updated_at = now()
WHERE id = p_user_id;

-- Increment held hexes if new ownership
IF p_capture_type IN ('neutral', 'steal') THEN
  UPDATE users SET current_held_hexes = current_held_hexes + 1 WHERE id = p_user_id;
  IF p_capture_type = 'steal' AND p_previous_owner IS NOT NULL THEN
    UPDATE users SET current_held_hexes = current_held_hexes - 1 WHERE id = p_previous_owner;
  END IF;
END IF;

-- Update streak if first capture today
PERFORM update_streak_if_needed(p_user_id);
END;
$$;

```

4.5 Realtime "hex stolen" notification

Use Supabase Realtime `postgres_changes` on `captures` table where `previous_owner_id = current_user_id`:

```

typescript

useEffect(() => {
  if (!currentUserId) return
  const sub = supabase.channel('hex-stolen')
    .on('postgres_changes', {
      event: 'INSERT',
      schema: 'public',
      table: 'captures',
      filter: `previous_owner_id=eq.${currentUserId}`
    }, payload => {
      showInAppToast(`🚨 Someone stole your hex!`)
      // Or fetch full detail and show banner
    })
    .subscribe()
  return () => { supabase.removeChannel(sub) }
}, [currentUserId])

```

4.6 PPH accrual job (every hour)

pg_cron job:

```
sql

SELECT cron.schedule('pph-hourly', '0 * * * *', $$
  UPDATE users u SET
    lifetime_points = lifetime_points + COALESCE(pph_sum.total, 0),
    current_round_points = current_round_points + COALESCE(pph_sum.total, 0)
  FROM (
    SELECT zo.user_id, SUM(h.pph_value) AS total
    FROM zone_ownership zo
    JOIN hexes h ON h.h3_index = zo.h3_index
    WHERE zo.pph_paused = FALSE
      AND zo.user_id IS NOT NULL
    GROUP BY zo.user_id
  ) pph_sum
  WHERE u.id = pph_sum.user_id;
$$);
```

Acceptance Criteria

- ☐ User can capture an unowned hex by standing in it for 20s
- ☐ Capture fails gracefully if user leaves hex, moves too fast, or accelerometer detects no movement
- ☐ User can steal an enemy hex (validated, 1.3x IP)
- ☐ Fresh Paint immunity prevents instant re-steal
- ☐ Block cooldown prevents capture for 15 min
- ☐ Revisit works after 23 hours, awards 50% IP
- ☐ Previous owner receives push notification on hex steal
- ☐ PPH ticks every hour for all held hexes
- ☐ Daily cap enforced (200 RP for L1)
- ☐ Soft cap at 50 hexes pauses PPH on new captures
- ☐ Hard cap at 200 hexes blocks new captures

What NOT to do

- **✗** Don't enable runs/paths yet (Phase 10)
- **✗** Don't add fortify yet (post-MVP)
- **✗** Don't add hex naming yet (Phase 6+)

PHASE 5 — Points, Levels & Safe Points (Week 6)

Goal: Level progression works, Safe Points prevent inactive coasting.

Tasks

5.1 Level-up logic

Postgres function that runs after every points update:

```
sql

CREATE OR REPLACE FUNCTION check_level_up(p_user_id UUID) RETURNS VOID LANGUAGE plpgsql
DECLARE
    v_lp BIGINT;
    v_current_level INT;
    v_new_level INT;
BEGIN
    SELECT lifetime_points, level INTO v_lp, v_current_level FROM users WHERE id = p_user_id;

    v_new_level := CASE
        WHEN v_lp >= 30000 THEN 5
        WHEN v_lp >= 10000 THEN 4
        WHEN v_lp >= 2500 THEN 3
        WHEN v_lp >= 500 THEN 2
        ELSE 1
    END;

    IF v_new_level > v_current_level THEN
        UPDATE users SET
            level = v_new_level,
            daily_cap_remaining = daily_cap_remaining + 1000, -- level-up bonus
            lifetime_points = lifetime_points + 1000
        WHERE id = p_user_id;

        -- Queue notification
        INSERT INTO notifications (user_id, type, title, body, data)
        VALUES (p_user_id, 'level_up',
            'Level up! 🎉',
            'You are now ' || level_name(v_new_level),
            jsonb_build_object('new_level', v_new_level));
    END IF;
END;
$$;
```

5.2 Daily reset job (5 AM IST)

```
sql
```

```

SELECT cron.schedule('daily-reset', '30 23 * * *', $$ -- 23:30 UTC = 5:00 IST
-- Reset daily caps based on level
UPDATE users SET daily_cap_remaining = CASE level
  WHEN 1 THEN 200 WHEN 2 THEN 400 WHEN 3 THEN 600
  WHEN 4 THEN 800 WHEN 5 THEN 1000
END,
daily_cap_reset_at = now();

-- Check Safe Points - demote inactive users
WITH inactive_users AS (
  SELECT u.id, u.level FROM users u
  WHERE u.level > 1
  AND NOT EXISTS (
    SELECT 1 FROM captures c
    WHERE c.user_id = u.id
    AND c.captured_at > now() - interval '3 days'
  )
  AND u.vacation_tokens_available = 0
)
UPDATE users SET
  level = level - 1,
  daily_cap_remaining = CASE level - 1
    WHEN 1 THEN 200 WHEN 2 THEN 400 WHEN 3 THEN 600 WHEN 4 THEN 800
  END
WHERE id IN (SELECT id FROM inactive_users);

-- Re-issue vacation tokens monthly
UPDATE users SET vacation_tokens_available = 1
WHERE EXTRACT(DAY FROM now()) AT TIME ZONE 'Asia/Kolkata' = 1;
$$);

```

5.3 Profile screen

Show on `/profile`:

- Avatar + username + display name
- Current level + name (Walker/Strider/...)
- Progress bar to next level
- Lifetime points, current round points
- Current streak (with flame)
- Held hexes count
- Daily cap remaining
- Medals earned (grid)

5.4 Level-up celebration screen

Modal that appears on level-up:

- Full-screen with animation
- "Level Up!" big text
- New level name
- New perks listed
- "Continue" button

Acceptance Criteria

- ☐ Level auto-increments at thresholds
 - ☐ Daily cap increases with level
 - ☐ Demotion works for 3+ days inactive (skipping vacation tokens)
 - ☐ Vacation token consumes correctly
 - ☐ Profile screen shows accurate data
-

PHASE 6 — Medals, Streaks & Gamification (Week 7)

Goal: Launch set of 8 medals, streak tracking with notifications, capture share card.

Tasks

6.1 Seed medals

`scripts/seed-medals.ts`:

```
typescript
```

```
const MEDALS = [
  {id: 'first_blood', name: 'First Blood', tier: 'bronze', lp_reward: 50,
    criteria_type: 'capture_count', criteria_threshold: 1},
  {id: 'week_one', name: 'Week One', tier: 'bronze', lp_reward: 100,
    criteria_type: 'streak_days', criteria_threshold: 7},
  {id: 'month_one', name: 'Month One', tier: 'silver', lp_reward: 500,
    criteria_type: 'streak_days', criteria_threshold: 30},
  {id: 'centurion', name: 'Centurion', tier: 'silver', lp_reward: 200,
    criteria_type: 'capture_count', criteria_threshold: 100},
  {id: 'insomniac', name: 'Insomniac', tier: 'bronze', lp_reward: 50,
    criteria_type: 'midnight_capture', criteria_threshold: 1},
  {id: 'monsoon_warrior', name: 'Monsoon Warrior', tier: 'silver', lp_reward: 300,
    criteria_type: 'rainy_captures', criteria_threshold: 25},
  {id: 'neighbourhood_king', name: 'Neighbourhood King', tier: 'gold', lp_reward: 500,
    criteria_type: 'pincode_top1_days', criteria_threshold: 7},
  {id: 'bridge_crosser', name: 'Bridge Crosser', tier: 'bronze', lp_reward: 100,
    criteria_type: 'special', criteria_threshold: 1}
]
```

6.2 Medal evaluator Edge Function

`supabase/functions/medal-check/index.ts`:

- Triggered after every capture
- Re-evaluates each medal the user doesn't yet have
- Awards medals that match
- Sends in-app notification + push for new medals

6.3 Streak tracker

Postgres function `update_streak_if_needed` called from `execute_capture`:

sql

```

CREATE OR REPLACE FUNCTION update_streak_if_needed(p_user_id UUID) RETURNS VOID LANGUAGE SQL
DECLARE
    v_last_capture TIMESTAMPTZ;
    v_streak INT;
    v_last_streak_day DATE;
BEGIN
    SELECT last_capture_at, current_streak INTO v_last_capture, v_streak FROM users WHERE id = p_user_id;
    v_last_streak_day := (v_last_capture AT TIME ZONE 'Asia/Kolkata')::DATE;
    -- If user already captured today, no change
    IF v_last_streak_day = (now() AT TIME ZONE 'Asia/Kolkata')::DATE THEN RETURN; END IF;
    -- If yesterday, increment
    IF v_last_streak_day = ((now() AT TIME ZONE 'Asia/Kolkata')::DATE - 1) THEN
        UPDATE users SET current_streak = current_streak + 1 WHERE id = p_user_id;
    ELSIF v_last_streak_day < ((now() AT TIME ZONE 'Asia/Kolkata')::DATE - 1) THEN
        -- Streak broken; check freeze
        IF (SELECT streak_freezes_available FROM users WHERE id = p_user_id) > 0 THEN
            UPDATE users SET streak_freezes_available = streak_freezes_available - 1 WHERE id = p_user_id;
            -- Streak preserved
        ELSE
            UPDATE users SET current_streak = 1 WHERE id = p_user_id;
        END IF;
    END IF;
    -- Update longest if needed
    UPDATE users SET longest_streak = GREATEST(longest_streak, current_streak) WHERE id = p_user_id;
END;
$$;

```

6.4 9 PM streak nudge

pg_cron at 3:30 PM UTC (9:00 PM IST):

```

sql

SELECT cron.schedule('streak-nudge', '30 15 * * *', $$
    INSERT INTO notifications (user_id, type, title, body)
    SELECT id, 'streak_at_risk',
        '🔥 Your ' || current_streak || '-day streak is at risk!',
        'Capture one hex before midnight to keep it alive.'
    FROM users
    WHERE current_streak >= 3
        AND (last_capture_at AT TIME ZONE 'Asia/Kolkata')::DATE < (now() AT TIME ZONE 'Asia/Kolkata')::DATE
        AND notification_prefs->>'streak' = 'true'
        AND banned_permanently = FALSE
$$);

```

6.5 Capture Share Card

`components/capture/ShareCard.tsx`:

- 1080x1080 image generated on-device via `react-native-view-shot`
- Shows: user avatar, neighbourhood, captures today, points, rank, streak, QR to app download
- Saffron + dark theme
- Auto-shared via `expo-sharing` (defaults to WhatsApp on Android)

typescript

```
async function shareCaptureCard(stats: DailyStats) {
  const cardUri = await captureRef(cardRef, {
    format: 'png', quality: 1, width: 1080, height: 1080
  })
  await Sharing.shareAsync(cardUri, {
    mimeType: 'image/png',
    dialogTitle: 'Share your captures'
  })
}
```

Acceptance Criteria

- ☐ All 8 launch medals defined and visible in profile
- ☐ Medal awarded immediately on trigger
- ☐ Streak updates correctly across IST day boundary
- ☐ Streak freeze consumed automatically on break
- ☐ Streak nudge fires at 9 PM IST to at-risk users
- ☐ Share card renders and shares to WhatsApp

PHASE 7 — Social: Friends, Leaderboards, Sharing (Week 8)

Goal: Add humans to the game. Without friends, nobody returns.

Tasks

7.1 Friends

- "Add friend" via username search
- Phone contacts sync (with permission)
- Pending/accepted/blocked states
- Friend list in `/friends` tab

Three views (toggle at top of `/leaderboard`):

- **Pincode:** users in your pincode, ranked by RP
- **City:** all Bangalore, top 100
- **Friends:** your friends only

Materialized view refreshed every 5 min:

```
sql

CREATE MATERIALIZED VIEW leaderboard_pincode AS
SELECT u.id, u.username, u.display_name, u.avatar_url, u.level, u.hex_colour,
       u.current_round_points, u.pincode,
       ROW_NUMBER() OVER (PARTITION BY u.pincode ORDER BY u.current_round_points DESC) AS rank
FROM users u
WHERE u.banned_permanently = FALSE AND u.ghost_mode = FALSE;

CREATE INDEX leaderboard_pincode_idx ON leaderboard_pincode(pincode, rank);

SELECT cron.schedule('refresh-leaderboard', '* /5 * * * *', $$
  REFRESH MATERIALIZED VIEW CONCURRENTLY leaderboard_pincode;
$$);
```

7.3 Hex steal banner

When friend steals your hex (via Realtime):

- In-app banner: "[Friend Name] stole your hex in [Neighbourhood]"
- Button: "Get it back"
- Tap → opens map centred on that hex

Acceptance Criteria

- ☐ Friend search, add, accept, block all work
- ☐ Contacts sync (with permission) suggests users
- ☐ All 3 leaderboard views render
- ☐ Steal banner appears for friends and strangers
- ☐ Ghost Mode hides you from leaderboards (women's safety MVP)

PHASE 8 — Anti-Cheat Layer (Week 9)

Goal: Lock down obvious cheating. Won't beat determined attackers — but kills 95% of casual

abuse.

Tasks

8.1 Mock location detection (Android)

In `lib/antiCheat/mockDetection.ts`:

```
typescript

import * as Location from 'expo-location'

export function isMockLocation(location: Location.LocationObject): boolean {
  if (Platform.OS === 'android') {
    return location.mocked === true
  }
  // iOS doesn't expose isMock natively; rely on App Attest
  return false
}
```

If `isMock = true`, instant rejection + strike.

8.2 Play Integrity API (Android)

Set up Play Integrity in Google Cloud Console; integrate `expo-application` + native module:

```
typescript

async function getIntegrityToken(): Promise<string> {
  if (Platform.OS !== 'android') return ''
  const token = await PlayIntegrity.requestIntegrityToken({
    cloudProjectNumber: GOOGLE_CLOUD_PROJECT_NUMBER,
    nonce: generateNonce()
  })
  return token
}
```

Server verifies token. Failed integrity → +50 risk score.

8.3 App Attest (iOS)

Similar setup with `DCAppAttestService`. Server verifies attestation.

8.4 Risk scorer

`lib/antiCheat/riskScorer.ts`:

```
typescript
```

```

export async function calculateRiskScore(input: {
  user_id: string,
  gps_trace: GpsPing[],
  accelerometer_steps: number,
  is_mock: boolean,
  integrity_token?: string,
  hex: Hex
}): Promise<number> {
  let score = 0

  if (input.is_mock) score += 60

  // Integrity
  if (!await verifyIntegrity(input.integrity_token, input.user_id)) score += 50

  // Speed
  const maxSpeed = Math.max(...input.gps_trace.map(p => p.speed_kmh ?? 0))
  if (maxSpeed > 10) score += 30
  if (maxSpeed > 25) score += 90 // auto-reject

  // Accelerometer
  if (input.accelerometer_steps === 0) score += 25

  // Impossible jump
  for (let i = 1; i < input.gps_trace.length; i++) {
    const dist = haversine(input.gps_trace[i-1], input.gps_trace[i])
    const dt = (new Date(input.gps_trace[i].timestamp).getTime() -
      new Date(input.gps_trace[i-1].timestamp).getTime()) / 1000
    if (dist > 100 && dt < 5) score += 100 // 100m in 5s = 72 km/h
  }

  // New account spam
  const userAge = await getUserAccountAgeDays(input.user_id)
  const todayCaptures = await getTodayCaptureCount(input.user_id)
  if (userAge < 1 && todayCaptures > 20) score += 20

  return Math.min(score, 100)
}

```

8.5 Three-strike system

Edge function tracks strikes and applies actions:

- Strike 1: Warning notification, 24h shadow-ban (capture works, leaderboard hidden)
- Strike 2: 7-day ban — captures rejected, app shows ban screen with appeal link

- Strike 3: Permaban + phone number added to `phone_blacklist` table

8.6 Manual review queue UI (founder dashboard)

Build a simple Next.js admin dashboard on Vercel free tier:

- Login with admin email
- View `anti_cheat_events WHERE reviewed = FALSE` ordered by risk_score DESC
- Buttons: "Confirm cheat" (strike), "False positive" (clear), "Open user profile"
- Daily review target: 30 minutes covers ~50 events

Acceptance Criteria

- ☐ Mock location captures rejected with strike
 - ☐ Play Integrity verified server-side
 - ☐ App Attest verified server-side
 - ☐ Risk score ≥ 80 auto-rejects
 - ☐ 3-strike escalation works end-to-end
 - ☐ Admin dashboard accessible
-

PHASE 9 — Polish, Onboarding & Beta (Week 10)

Goal: Production-ready closed beta. 25 friends. Bug-free. Battery-tuned.

Tasks

9.1 Onboarding flow

Post-OTP, before main app:

- Screen 1: "Welcome to Hexa — capture your city" + animated hex map
- Screen 2: Permission request — foreground location
- Screen 3: "Set your home" → uses GPS to detect pincode
- Screen 4: "Are you here to walk, compete, or conquer?" → sets UI depth
- Screen 5: First-walk tutorial — places a glowing hex 30m from user, "Walk to it"

9.2 Battery profiling

Use Android Studio Battery Profiler + Xcode Instruments:

- Target: $< 8\%$ battery / hour during active walk
- Target: $< 1\%$ battery / hour idle in background
- If above thresholds, switch to `react-native-background-geolocation` (transistorsoft, \$250 licence)

9.3 In-app feedback button

Floating button in profile → opens DM to founder's WhatsApp / Telegram with pre-filled user ID + app version.

9.4 Push notification permission

Request only after first successful capture (timing matters — they're hooked, more likely to grant).

9.5 TestFlight + Play Internal Testing

- Build with EAS Build production profile
- Upload to TestFlight + Play Internal
- Invite 25 beta testers via emails
- Slack/WhatsApp group for bug reports

9.6 Bug bash

7-day bug bash:

- Daily 30-min triage
- Top 20 bugs fixed
- Performance issues addressed

Acceptance Criteria

- ☐ 25 beta testers active for 7 days
 - ☐ Crash rate < 1%
 - ☐ Battery drain < 8%/hour during active walks
 - ☐ All P0 bugs fixed
 - ☐ App Store + Play Store metadata + screenshots ready
-

PHASE 10 — Paths System (Month 3)

Goal: Add the runner mechanic without breaking hexes.

Tasks

10.1 Hand-curate 30 launch paths

Manually walk and record GPS traces for ~30 paths across HSR, Indiranagar, Koramangala. Examples:

- Agara Lake Loop (2.4 km)

- 100 Feet Stretch (1.8 km)
- Cubbon Morning Mile (1.6 km)
- Sankey Tank Round (1.7 km)
- HSR BDA Park Path (800m)

For each:

- Define 5-15 checkpoints (lat/lng) at ~150m intervals
- Walking time cap = $\text{distance_km} \times 12 \text{ min/km}$
- Running time cap = $\text{distance_km} \times 6 \text{ min/km}$
- IP = $5 \times$ the average hex IP along the path
- PPH = $\text{sum of underlying hex PPH} \times 1.5$

10.2 Path capture flow

UI: tap path on map → "Start attempt" → live tracker shows next checkpoint + remaining time

- Each checkpoint requires 10s dwell + sequence preserved
- Speed cap on entire path: $\text{avg} \leq 9 \text{ km/h}$ sustained
- Successful completion → path owned, all underlying hexes auto-paint
- Steal by completing faster than current owner's best time

10.3 Path leaderboard

- Per-path leaderboard of best completion times
- Strava-style segment ranking

Acceptance Criteria

- ☐ 30 paths defined and active
 - ☐ Path capture validates checkpoint sequence + speed cap
 - ☐ Path steal by faster time works
 - ☐ Underlying hexes auto-paint while path held
-

PHASE 11 — Clans (Month 4)

Goal: Apartment/college/tech park identity. The killer retention feature.

Tasks

- Clan creation gated to L3+
- Verification via RWA admin email or college .edu domain

- Inter-clan leaderboards per neighbourhood
- Clan chat (use Supabase Realtime channels)
- Clan tag prefix in username displays
- Weekly clan war: highest collective RP wins clan medal

Acceptance Criteria

- ☐ Users can create + join clans
 - ☐ Clan leaderboard works
 - ☐ Clan war weekly cycle automated
-

PHASE 12 — Brand Bazaar & Monetization (Month 4-5)

Goal: First ₹ of revenue. Brand vouchers redeemable for RP/LP.

Tasks

- Bazaar tab in profile area
- 5 launch brand partners (Third Wave, Subko, Cult, Decathlon, Blue Tokai)
- Voucher schema: brand, denomination, RP cost, stock, expiry
- Redemption flow: spend RP → get UPI voucher code via Razorpay X / Cashfree
- Brand admin dashboard (separate Next.js admin app)

Acceptance Criteria

- ☐ 5 brand partners contracted
 - ☐ Voucher redemption end-to-end works
 - ☐ First brand revenue received
-

Appendix A — Capture Flow Pseudocode

```
on user_enters_hex(hex):
  if hex.owner_id == current_user.id:
    state = IN_HEX_OWN
    show "Your hex - visiting resets decay timer"
    return

  if hex.fresh_paint_until > now():
    state = IN_HEX_FRESH_PAINT
    show "🎨 Fresh Paint - wait until HH:MM"
    return
```

```
if hex.block_cooldown_until > now():
    state = IN_HEX_COOLDOWN
    show "Cooldown – wait HH:MM"
    return

if hex.owner_id != null:
    state = IN_HEX_HOSTILE
    show "Steal this hex (held by [owner_username]) – tap Capture"
else:
    state = IN_HEX_NEUTRAL
    show "Unclaimed hex – tap Capture"

user_taps_capture()
state = DWELLING
start_dwell_timer(20s)

on dwell_timer_tick(second):
    if !point_in_hex(current_gps, hex):
        reset_dwell_timer() # Started over
        return

    if current_gps.accuracy > 25:
        pause_dwell_timer()
        show "Improve GPS signal"
        return

    if current_gps.speed_kmh > 10:
        pause_dwell_timer()
        show "Slow down"
        return

    decrement_remaining_seconds()

on dwell_timer_complete():
    state = CAPTURING
    show "Capturing..."

    payload = {
        h3_index: hex.h3_index,
        gps_trace: last_30_seconds_of_gps_pings,
        accelerometer_steps: step_count_during_dwell,
        device_integrity_token: await get_integrity_token(),
        is_mock: last_gps.mocked
    }

    result = await POST /validate-capture { payload }
```

```
if result.success:
    state = CAPTURED
    play_celebration_animation()
    play_capture_sound()
    haptic_feedback()
    update_local_state(hex.h3_index, owner=current_user.id)
    show "+{result.ip_awarded} IP"
    after 2 seconds, state = IN_HEX_OWN

else:
    state = FAILED
    show "Capture failed: {result.error}"
    after 2 seconds, state = IN_HEX_NEUTRAL # or whatever appropriate
```

Appendix B — Hex Pre-Generation Script

typescript

```
// scripts/generate-hexes.ts
import { polygonToCells, cellToBoundary, cellToLatLng, isValidCell } from 'h3-js'
import { createClient } from '@supabase/supabase-js'
import * as fs from 'fs'

const supabase = createClient(
  process.env.SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)

const BANGALORE_BOUNDS = {
  north: 13.1737,
  south: 12.8340,
  east: 77.7798,
  west: 77.4601
}

async function generate() {
  console.log('Generating H3 hexes for Bangalore at resolution 10...')

  // Define polygon as GeoJSON [[lng, lat], ...]
  const polygon = [
    [BANGALORE_BOUNDS.west, BANGALORE_BOUNDS.south],
    [BANGALORE_BOUNDS.east, BANGALORE_BOUNDS.south],
    [BANGALORE_BOUNDS.east, BANGALORE_BOUNDS.north],
    [BANGALORE_BOUNDS.west, BANGALORE_BOUNDS.north],
    [BANGALORE_BOUNDS.west, BANGALORE_BOUNDS.south]
  ]

  // Note: h3-js uses [lat, lng] for cellToLatLng but polygonToCells expects [lng, lat]
  const cells = polygonToCells(polygon, 10, true)
  console.log(`Generated ${cells.length} hexes`)

  // Insert in batches of 1000
  const BATCH = 1000
  for (let i = 0; i < cells.length; i += BATCH) {
    const batch = cells.slice(i, i + BATCH).map(cell => {
      const [lat, lng] = cellToLatLng(cell)
      const boundary = cellToBoundary(cell, true) // [lng, lat]
      const geojson = {
        type: 'Polygon',
        coordinates: [boundary]
      }
      return {
        h3_index: BigInt('0x' + cell).toString(),
        centroid_lat: lat,
```

```
        centroid_lng: lng,
        boundary_geojson: geojson,
        rarity: 1,
        ip_value: 100,
        pph_value: 3
      }
    })

    const { error } = await supabase.from('hexes').upsert(batch, { onConflict: 'h3_index' })
    if (error) {
      console.error(`Batch ${i} failed:`, error)
      break
    }
    console.log(`Inserted ${i + batch.length}/${cells.length}`)
  }

  console.log('Done.')
}

generate().catch(console.error)
```

Appendix C — Battery & Performance Targets

Metric	Target	Stretch
Battery / hour active walk	< 8%	< 5%
Battery / hour idle background	< 1%	< 0.5%
App cold start	< 3s	< 2s
Map first paint	< 1.5s	< 1s
Hex render frame rate	60fps	60fps
Capture validation latency	< 500ms	< 200ms
Crash rate	< 1%	< 0.3%
App size (APK/IPA)	< 50MB	< 30MB

Appendix D — Decisions Still Pending

These decisions are intentionally deferred. Resolve them when you reach the relevant phase:

- 1. **Pincode polygon dataset** — Will you use datameet/india-pincodes or commercial dataset? (Phase 3)
- 2. **Strava integration** — Will you support importing Strava runs for path attempts? (Phase 10)
- 3. **WhatsApp Business API vs Cloud API** — For automated leaderboard share cards (Post-MVP)
- 4. **Razorpay X vs Cashfree** — For brand-funded UPI payouts (Phase 12)
- 5. **Admin dashboard hosting** — Vercel free, or self-host on the same Hetzner box? (Phase 8)
- 6. **Multi-language strings** — i18next or react-i18next? (Phase 6+)
- 7. **Custom map style** — Stick with MapTiler default, or design a custom hexa-themed map? (Phase 9 polish)
- 8. **Apple Watch / WearOS** — Build companion app? (Post-PMF only)
- 9. **Referral bonus structure** — +20 RP per friend who walks (StepSetGo) or different? (Phase 7)
- 10. **Pro subscription pricing** — ₹99/mo or ₹149/mo? Should it gate clan creation? (Phase 12+)

EXECUTIVE TIMELINE SUMMARY

Week	Phase	Goal
0 (Day 0-3)	Phase 0	Accounts, repo, decisions locked
1	Phase 1	Auth (phone OTP) + basic shell
2	Phase 2	Map + hex rendering
3	Phase 3	GPS + 520K hex pre-generation
4-5	Phase 4	Capture mechanics end-to-end
6	Phase 5	Levels + Safe Points
7	Phase 6	8 medals + streaks + share card
8	Phase 7	Friends + leaderboards
9	Phase 8	Anti-cheat layer
10	Phase 9	Polish + closed beta (25 users)
11-13	Phase 10	Public launch in HSR + Paths
14-17	Phase 11	Clans + 500 users

Week	Phase	Goal
18-24	Phase 12	Brand Bazaar + monetization + 1000 users

End of Month 6 decision gate: MAU + retention + brand deals → go all in / iterate / kill.

NOTES FOR CLAUDE CODE

When implementing each phase:

1. **Always read this entire phase section before starting work.** Don't jump to code.
2. **Use TypeScript strict mode.** Run `tsc --noEmit` before committing.
3. **Write unit tests for core logic** — capture FSM, point calculator, anti-cheat risk scorer, hex utilities. Skip UI tests in MVP.
4. **Commit small, descriptive commits.** Use Conventional Commits: `feat(capture): add dwell timer`, `fix(map): handle empty viewport`, etc.
5. **Ask before adding dependencies.** Don't auto-install random libraries.
6. **Ask before structural changes** to schema, FSM states, or critical game logic.
7. **All times are IST (Asia/Kolkata).** Default everywhere.
8. **All currencies INR.** Never USD in user-facing strings.
9. **Comment WHY, not WHAT.** Code shows what; comments explain the game design reason.
10. **At end of each phase, run the acceptance checklist explicitly.** Don't proceed if any item fails.

END OF BUILD DOCUMENT

Total length: ~14,000 words. This is your complete spec. Build incrementally, ship beta at Week 10, validate with 1,000 users by Week 22, decision at Month 6. Good luck.